

Progetto Informatica con Laboratorio

Equazione del calore 2D e riordinamento nelle fattorizzazioni Cholesky

Docenti:

Roberto Grossi Leonardo Robol

Studentessa:

Giulia Casti

Anno Accademico 2025/2026

1 Introduzione

Il presente documento descrive lo sviluppo e l'evoluzione software del modulo `HeatEquationSolver`, progettato per la risoluzione numerica dell'equazione del calore 2D e lo studio del riordinamento delle matrici sparse nelle fattorizzazioni di Cholesky.

2 Modulo: HeatEquationSolver

Lo sviluppo del solutore procede in modo modulare ed evolutivo. Di seguito vengono dettagliati i singoli sottomoduli.

2.1 Sottomodulo 1: Generazione della griglia del dominio

2.1.1 Specifiche e Funzionalità

Il componente si occupa della discretizzazione spaziale del dominio di calcolo.

- **Input:** Intero N che determina la discretizzazione uniforme del dominio quadrato $[0, 1]^2$.
- **Output:** Griglia 2D rappresentata come insieme di punti.

La griglia è definita come un insieme di punti dove ogni elemento è identificato da una coppia di coordinate (x, y) . La generazione include tutti i punti del dominio, compreso il bordo, con passo spaziale $h = \frac{1}{N}$, per un totale di $(N + 1) \times (N + 1)$ punti.

2.1.2 Strutture Dati

Per la rappresentazione in memoria delle entità coinvolte si adottano le seguenti scelte architetturali:

- **Punto:** Struttura (`struct`) composta da due variabili di tipo `double` per le coordinate x e y .

- **Griglia:** Array dinamico (`vector`) per la gestione della collezione di punti.

2.1.3 Complessità Attesa

La generazione della griglia richiede di iterare su tutti i punti del dominio discretizzato per un totale di $(N + 1)^2$ elementi. La complessità temporale attesa è dunque $O(N^2)$, poiché il calcolo delle coordinate di ciascun punto viene eseguito in tempo costante $O(1)$.

2.2 Sottomodulo 2: Esportazione dei dati su file

2.2.1 Specifiche e Funzionalità

Questo componente gestisce la persistenza su disco delle informazioni inerenti al dominio e alla struttura del grafo associato alla discretizzazione.

- **Input:** Griglia di punti, lista degli archi, lista dei nodi.
- **Output:** File di testo su memoria secondaria.

Nello specifico, il sottomodulo mette a disposizione le funzionalità necessarie per:

- La generazione e scrittura di un file contenente la lista dei soli punti interni della griglia (escludendo i punti appartenenti al bordo).
- La generazione di un file contenente la lista degli archi che compongono il grafo.
- La generazione di un file contenente la lista dei nodi del grafo.

2.2.2 Strutture Dati

Per la fase di esportazione non si rendono necessarie strutture dati complesse aggiuntive in RAM. L'operazione richiede esclusivamente l'apertura e la gestione di

canali di comunicazione con lo spazio di I/O (stream di output su file) per la scrittura sequenziale dei dati immagazzinati nelle strutture preesistenti.

2.2.3 Complessità Attesa

Le operazioni di scrittura su memoria di massa richiedono l'attraversamento sequenziale delle strutture dati ricevute in input. La complessità temporale attesa è $O(V + E) = O(N^2)$, in quanto la scrittura di ogni nodo e di ogni arco nel relativo stream di output richiede un numero di operazioni proporzionale alla dimensione del grafo.

2.3 Sottomodulo 3: Creazione del grafo di adiacenza

2.3.1 Specifiche e Funzionalità

Il componente si occupa della modellizzazione topologica dei nodi interni mediante la costruzione di un grafo non orientato.

- **Input:** Intero N che determina la discretizzazione uniforme del dominio quadrato $[0, 1]^2$.
- **Output:** Grafo non orientato $G = (V, E)$.

Il grafo modellato soddisfa le seguenti caratteristiche:

- Ogni nodo in V rappresenta un punto interno della griglia (escludendo i nodi di bordo).
- Ogni arco in E rappresenta una relazione di adiacenza orizzontale o verticale tra punti vicini (struttura a 5 punti).
- Ogni nodo possiede coordinate geometriche (x, y) associate.

Le funzionalità richieste per questo sottomodulo includono:

- Generazione dei soli punti interni della griglia.
- Assegnazione di identificatori numerici progressivi ai nodi.
- Costruzione delle relazioni di adiacenza tra nodi adiacenti.
- Esportazione dei dati nei file `coords.txt` e `connectivity.txt`.
- Supporto a successivi riordinamenti dei nodi (es. indicizzazione per algoritmi di fill-in reduction).

2.3.2 Strutture Dati

Per rappresentare la topologia del grafo in modo efficiente si suggeriscono:

- **Liste di adiacenza:** Implementate tramite `std::vector<std::vector<int>>` per garantire una rappresentazione compatta ed efficiente del grafo sparso.
- **Mappatura geometrica/topologica:** Strutture dati ausiliarie (come hash map o array bidimensionali di indicizzazione) per la conversione bidirezionale tra coordinate geometriche (i, j) e identificatori numerici progressivi dei nodi.

2.3.3 Complessità Attesa

La costruzione del grafo e la determinazione delle adiacenze richiedono una complessità temporale attesa pari a $O(|V|) = O((N - 1)^2) = O(N^2)$, poiché il grado di ciascun nodo è limitato superiormente da 4 e l'identificazione dei vicini viene eseguita in tempo costante.

2.4 Sottomodulo 4: Costruzione dell'ordinamento dei nodi

2.4.1 Specifiche e Funzionalità

Questo sottomodulo implementa la tecnica di riordinamento dei nodi del grafo per ridurre il fenomeno del *fill-in* durante la successiva fattorizzazione di Cholesky della matrice associata.

- **Input:** File `coords.txt` (coordinate dei nodi) e `connectivity.txt` (topologia degli archi del grafo).
- **Output:** File contenente la sequenza ordinata degli identificatori dei nodi (permutazione).

Le funzionalità richieste comprendono:

- Lettura ed elaborazione del grafo di adiacenza a partire dai file d'ingresso.
- Riordinamento dei nodi tramite una procedura ricorsiva basata su strategie di partizionamento (es. Dissecazione Annidata / *Nested Dissection*).
- Identificazione ricorsiva di separatori di vertici che dividono il grafo in sottografi disgiunti, numerando prima i sotto-problemi e infine il separatore.

2.4.2 Strutture Dati

Per supportare la procedura ricorsiva di riordinamento si suggeriscono:

- **Permutazione dei nodi:** Un `std::vector<int>` per memorizzare l'ordine finale dei nodi.

- **Mappatura dei sottografi:** Strutture ausiliarie (come vettori di flag booleani `std::vector<bool>` o sottoliste di vertici) per tracciare i nodi attivi in ciascuna chiamata ricorsiva.
- **Coda / Pila ausiliaria:** Strutture dati di supporto (`std::queue` o `std::vector`) per la ricerca e l'individuazione del separatore di vertici (es. tramite viste BFS/RCM).

2.4.3 Complessità Attesa

La procedura ricorsiva di dissecazione divide il dominio ad ogni livello. La complessità temporale attesa dipende dal costo del partizionamento ad ogni livello di ricorsione $d = O(\log N)$:

- Il costo per trovare un separatore su un sottografo con V' nodi è proporzionale alle dimensioni del sottografo stesso, ossia $O(V')$.
- Risolvendo la relazione di ricorrenza $T(V') = 2T(V'/2) + O(V')$, si ottiene una complessità temporale complessiva attesa di $O(|V| \log |V|) = O(N^2 \log N)$.